

CS306 - Database Systems

Term Project Phase 3 Report

Project Title: Gym Management System

Group 22 Members:

- Hamza Eren İnan (34502)
- Alperen Sarışen (33886)

Brief Description of the Database

The Gym Management System is a relational database designed to manage the daily operations of a fitness center network. It tracks multiple gym branches and the employees working within them, including their specific job titles and salaries. The system manages gym members, their personal information, and their emergency contacts. Additionally, it keeps an inventory of the equipment located in each branch and schedules fitness classes led by trainers, allowing members to enroll in specific classes.

Queries Written by Hamza Eren İnan:

Query 1 – Description

Retrieve all rows and columns from the Branch table to see all facility locations.

(Category A - Retrieve all rows)

SQL script

```
SELECT * FROM Branch;
```

Relational Algebra Version

Branch

Query 2 – Description

List the names and phone numbers of employees who work in the branch with ID 1.

(Category B - Join with projection of selected columns)

SQL script

```
SELECT E.ename, B.bphone FROM Employee E, Branch B WHERE E.bid = B.bid AND B.bid = 1;
```

Relational Algebra Version

$\pi_{\text{ename}, \text{bphone}} (\sigma_{\text{bid}=1} (\text{Employee} \bowtie_{\text{Employee.bid}=\text{Branch.bid}} \text{Branch}))$

Query 3 – Description

Find the maximum salary among all employees in the gym system. (Category C - Aggregate MAX)

SQL script

```
SELECT MAX(salary)
FROM Employee;
```

Relational Algebra Version $\mathcal{G}_{\text{max}}(\text{salary})$ (Employee)

Query 4 – Description

List all members, sorted by their age in ascending order. (Category D - ORDER BY ascending)

SQL script

```
SELECT *
FROM Member
ORDER BY m_age ASC;
```

Relational Algebra Version $\tau_{\text{m_age}}$ (Member)

Query 5 – Description

Find branch IDs that have more than one employee by grouping results and filtering. (Category E - Filtering grouped results with HAVING)

SQL script

```
SELECT bid, COUNT()
FROM Employee
GROUP BY bid HAVING COUNT() > 1;
```

Relational Algebra Version $\sigma_{\text{count} > 1}(\text{bid } \mathcal{G}_{\text{count}}(*) \text{ as count (Employee)})$

Queries Written by Alperen Sarişen

Query 6 – Description

Show the names of employees who have a salary greater than 12000.

(Category A - Selection + Projection together)

SQL script:

```
SELECT ename
```

```
FROM Employee
```

```
WHERE salary > 12000;
```

Relational Algebra Version $\pi_{\text{ename}} (\sigma_{\text{salary} > 12000} (\text{Employee}))$

Query 7 – Description

List the names of members and the names of the classes they are enrolled in.

(Category B - Join between 3 tables)

SQL script

```
SELECT M.mname, C.cname
```

```
FROM Member M, Enrolled En, Class C
```

```
WHERE M.mid = En.mid AND En.cid = C.cid;
```

Relational Algebra Version $\pi_{\text{mname}, \text{cname}} (\text{Member} \bowtie_{\text{Member.mid}=\text{Enrolled.mid}} \text{Enrolled} \bowtie_{\text{Enrolled.cid}=\text{Class.cid}} \text{Class})$

Query 8 – Description

Calculate the average age of all gym members. (Category C - Aggregate AVG)

SQL script

```
SELECT AVG(mage) FROM Member;
```

Relational Algebra Version $\mathcal{G}_{\text{avg(mage)}} (\text{Member})$

Query 9 – Description

List the names of all fitness classes, sorted alphabetically in descending order. (Category D - ORDER BY descending)

SQL script

```
SELECT cname  
FROM Class  
ORDER BY cname DESC;
```

Relational Algebra Version $\tau_{\text{cname desc}}(\pi_{\text{cname}}(\text{Class}))$

Query 10 – Description

Find the total number of enrollments for each fitness class by joining the tables and grouping. (Category E - Join + Group By)

SQL script

```
SELECT C.cname, COUNT(En.mid)  
FROM Class C  
LEFT JOIN Enrolled En ON C.cid = En.cid GROUP BY C.cname;
```

Relational Algebra Version $\text{cname } \mathcal{G}_{\text{count}(\text{mid})}(\text{Class} \bowtie_{\text{Class.cid=Enrolled.cid}} \text{Enrolled})$

Appendix - AI Usage Documentation

During the development of this phase, AI tools were utilized strictly for limited, supportive tasks. We document our AI usage as follows:

- **Whether AI tools were used:** Yes, strictly for generating mock data (to populate the tables) and for verifying Relational Algebra syntax.
- **The prompts used:** "Can you generate 5 random Turkish names and ages for my database, and list 5 real-world gym names operating in Turkey (e.g., MACFit)? Also, check if my Relational Algebra syntax for a subquery and grouping is formatted correctly in LaTeX."
- **The AI responses received:** The AI provided a list of randomly generated member/trainer names (Ali, Ayşe, vb.), real gym franchise names (MACFit, Jatomi Fitness, Hillside City Club, Carrera Fitness, Sports International), and confirmed the standard LaTeX notation (like γ for aggregate and $\bowtie$ for join) for relational algebra expressions.

- **How the output was used:** The generated names, numerical values, and gym names were used directly in our INSERT statements to populate our database with realistic test data. The relational algebra syntax checks were reviewed and applied to format our mathematical expressions correctly in the report. All core database designs, logical query structures, and SQL formulations were developed entirely independently by the project members.